

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Applied business Analyst

Module 2

Practical - X

Roll No : T018	Name: Sachin Ramesh Mehta
Class:TYIT	Batch:1
Date of Assignment:03/10/20206	Date of Submission: 03/10/20206

10. Regression Analysis and Time Series Analysis:

a. Develop a regression model to predict future sales based on historical data, assess model performance, and interpret the significance of predictor variables.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import r2_score, mean_squared_error
```

```
# Q1. Load the sales_data.csv dataset
```

```
df = pd.read_csv("sales_data.csv")
```

```
print("Dataset loaded successfully!")
```

```
# Q2. Display the first five records
```

```
print("\nFirst Five Records:")
```

```
print(df.head())
```

```
First Five Records:
   advertising  sales
0             5     22
1             8     28
2            10     31
3            12     35
4            15     40
```

```
# Q3. Identify independent and dependent variables
```

```
print("\nVariables:")
```

```
print("Independent Variable: advertising")
```

```
print("Dependent Variable: sales")
```

```
Variables:
Independent Variable: advertising
Dependent Variable: sales
```

```
# Q4. Select advertising as independent variable and sales as dependent variable
```

```
X = df[["advertising"]]
```

```
y = df["sales"]
```

```
print("\nIndependent Variable:")
```

```
print(X.head())
```

```
print("\nDependent Variable:")
```

```
print(y.head())
```

```
# Q5. Check data for missing values
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

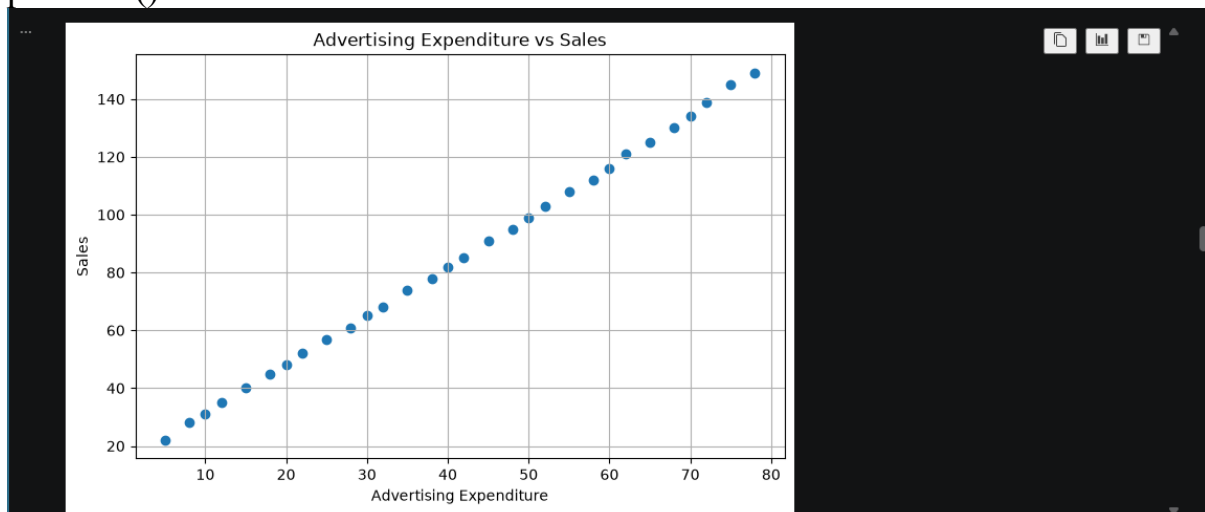
```
if df.isnull().sum().sum() == 0:
    print("No missing values found.")
else:
    print("Missing values are present.")
```

```
Independent Variable:
advertising
0      5
1      8
2     10
3     12
4     15

Dependent Variable:
0     22
1     28
...
advertising    0
sales          0
dtype: int64
No missing values found.
```

```
# Q6. Create scatter plot
plt.figure(figsize=(8, 5))
```

```
plt.scatter(
    df["advertising"],
    df["sales"]
)
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")
plt.title("Advertising Expenditure vs Sales")
plt.grid(True)
plt.show()
```



```
# Q7. Develop a simple linear regression model
model = LinearRegression()
model.fit(X, y)
print("\nLinear Regression Model created successfully.")
```

Q8. Display the regression equation

```
b0 = model.intercept_
```

```
b1 = model.coef_[0]
```

```
print("\nRegression Equation:")
```

```
print(f'Sales = {b0:.2f} + ({b1:.2f} × Advertising)')
```

```
...
Linear Regression Model created successfully.

Regression Equation:
Sales = 13.49 + (1.72 × Advertising)

Prediction:
Advertising Expenditure = 60
Predicted Sales = 116.88
```

Q9. Predict sales for a given advertising expenditure

```
advertising_value = 60
```

```
predicted_sales = model.predict([[advertising_value]])
```

```
print("\nPrediction:")
```

```
print("Advertising Expenditure =", advertising_value)
```

```
print("Predicted Sales =", round(predicted_sales[0], 2))
```

```
Prediction:
Advertising Expenditure = 60
Predicted Sales = 116.88
```

Q10. Calculate R2 score

```
y_pred = model.predict(X)
```

```
r2 = r2_score(y, y_pred)
```

```
print("\nR2 Score:")
```

```
print(round(r2, 4))
```

```
R2 Score:
0.9995
```

Q11. Calculate Mean Squared Error

```
mse = mean_squared_error(y, y_pred)
```

```
print("\nMean Squared Error:")
```

```
print(round(mse, 4))
```

```
Mean Squared Error:
0.6601
```

Q12. Compare actual sales with predicted sales

```
comparison = pd.DataFrame({
```

```
    "Advertising": df["advertising"],
```

```
    "Actual Sales": y,
```

```
    "Predicted Sales": y_pred.round(2)
```

```
})
```

```
print("\nActual vs Predicted Sales:")
```

```
print(comparison)
```

Actual vs Predicted Sales:

	Advertising	Actual Sales	Predicted Sales
0	5	22	22.10
1	8	28	27.27
2	10	31	30.72
3	12	35	34.17
4	15	40	39.34
5	18	45	44.51
6	20	48	47.95
...			
26	70	134	134.12
27	72	139	137.56
28	75	145	142.73
29	78	149	147.90

Output is truncated. View as a scrollable element or open in a new window. Adjust all output settings.

Q13. Plot regression line with actual data points

```
plt.figure(figsize=(8, 5))
```

```
# Actual data points
```

```
plt.scatter(
    df["advertising"],
    df["sales"],
    label="Actual Data"
)
```

```
# Regression line
```

```
plt.plot(
    df["advertising"],
    y_pred,
    label="Regression Line"
)
```

```
plt.xlabel("Advertising Expenditure")
```

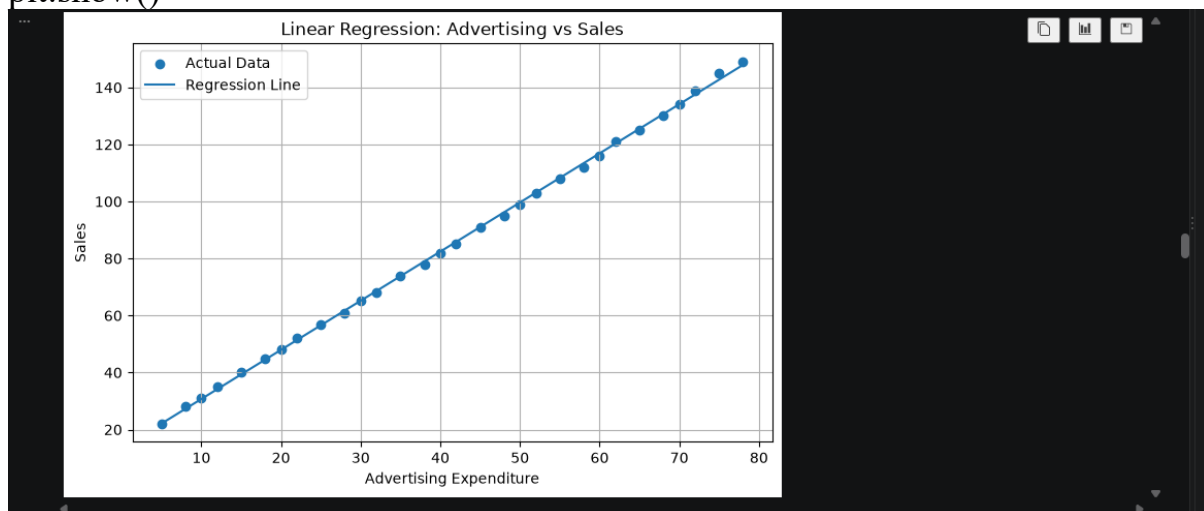
```
plt.ylabel("Sales")
```

```
plt.title("Linear Regression: Advertising vs Sales")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```



Q14. Interpret the regression results

```

print("\nRegression Interpretation:")
print("-----")
print("Intercept (b0):", round(b0, 2))
print("Slope (b1):", round(b1, 2))
print("R2 Score:", round(r2, 4))
print("MSE:", round(mse, 4))

```

```

if b1 > 0:
    print("\nThere is a positive relationship between")
    print("advertising expenditure and sales.")
    print("As advertising expenditure increases,")
    print("sales tend to increase.")
else:
    print("\nThere is a negative relationship between")
    print("advertising expenditure and sales.")

```

```

Regression Interpretation:
-----
Intercept (b0): 13.49
Slope (b1): 1.72
R2 Score: 0.9995
MSE: 0.6601

```

Q15. Conclusion

```

print("\nConclusion:")
print("The regression analysis shows that advertising")
print("expenditure has a positive relationship with sales.")
print("Higher advertising expenditure is associated")
print("with higher sales in the given dataset.")

```

```

There is a positive relationship between
advertising expenditure and sales.
As advertising expenditure increases,
sales tend to increase.

```

```

print("\nPART A COMPLETED SUCCESSFULLY!")

```

b. Apply time series analysis techniques (e.g., ARIMA, exponential smoothing) to forecast future demand for a product or service, and evaluate the accuracy of the forecasts.

```

Conclusion:
The regression analysis shows that advertising
expenditure has a positive relationship with sales.
Higher advertising expenditure is associated
with higher sales in the given dataset.

```

```

PART A COMPLETED SUCCESSFULLY!

```

PRACTICAL 10 - PART B

TIME SERIES ANALYSIS

Dataset: monthly_sales.csv

```

import pandas as pd

```

```

import numpy as np

```

```

import matplotlib.pyplot as plt

```

```

from sklearn.metrics import mean_squared_error

```

```

from statsmodels.tsa.holtwinters import ExponentialSmoothing

```

```
from statsmodels.tsa.arima.model import ARIMA
```

```
# -----
```

```
# Q16. Load monthly sales data
```

```
monthly_df = pd.read_csv("monthly_sales.csv")
```

```
print("Monthly Sales Data:")
```

```
print(monthly_df)
```

```
... Monthly Sales Data:
      Month  Sales
0  2025-01-01    120
1  2025-02-01    135
2  2025-03-01    128
3  2025-04-01    145
4  2025-05-01    150
5  2025-06-01    160
6  2025-07-01    155
7  2025-08-01    170
8  2025-09-01    165
9  2025-10-01    180
10 2025-11-01    190
11 2025-12-01    200
```

```
# Q17. Convert date column into date format
```

```
monthly_df["Month"] = pd.to_datetime(monthly_df["Month"])
```

```
# Set Month as index
```

```
monthly_df.set_index("Month", inplace=True)
```

```
print("\nDate Column Converted:")
```

```
print(monthly_df)
```

```
print("\nData Types:")
```

```
print(monthly_df.dtypes)
```

```
      Sales
Month
2025-01-01    120
2025-02-01    135
2025-03-01    128
2025-04-01    145
2025-05-01    150
2025-06-01    160
2025-07-01    155
```

```
# Q18. Plot monthly sales using a line chart
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(
    monthly_df.index,
    monthly_df["Sales"],
    marker="o"
)
```

```
plt.xlabel("Month")
```

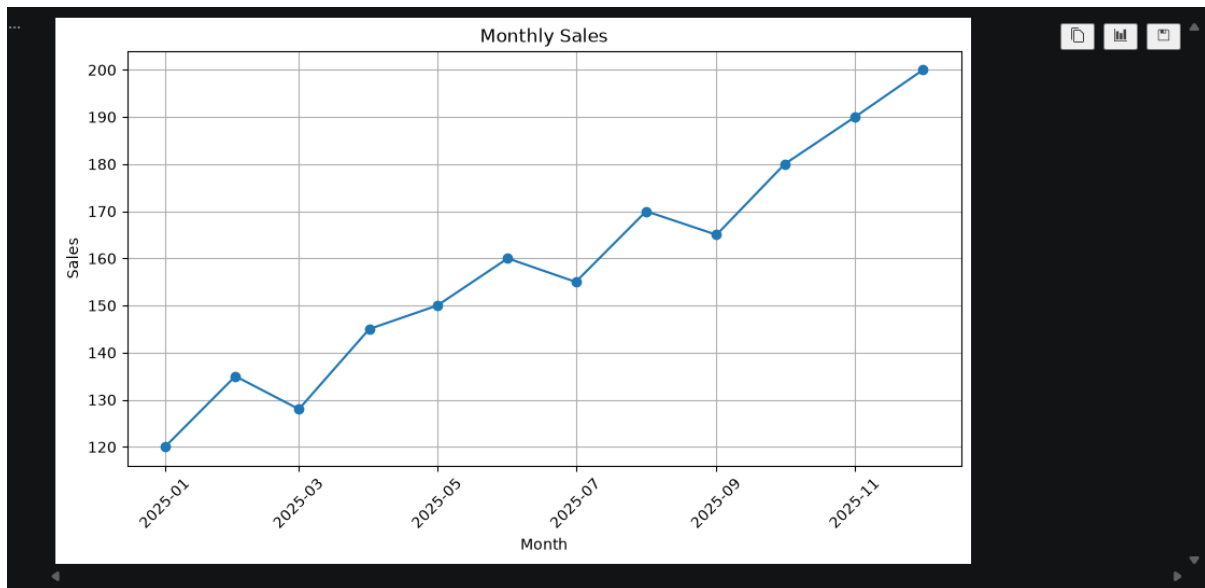
```
plt.ylabel("Sales")
```

```
plt.title("Monthly Sales")
```

```
plt.grid(True)
```

```
plt.xticks(rotation=45)
```

```
plt.show()
```



Q19. Observe the trend

```
first_sales = monthly_df["Sales"].iloc[0]
last_sales = monthly_df["Sales"].iloc[-1]
print("\nTrend Analysis:")
if last_sales > first_sales:
    print("The overall sales trend is increasing.")
elif last_sales < first_sales:
    print("The overall sales trend is decreasing.")
else:
    print("The sales trend is stable.")
print("First Month Sales:", first_sales)
print("Last Month Sales:", last_sales)
```

```
...
Trend Analysis:
The overall sales trend is increasing.
First Month Sales: 120
Last Month Sales: 200
```

Q20. Apply Exponential Smoothing

```
sales_series = monthly_df["Sales"]
exp_model = ExponentialSmoothing(
    sales_series,
    trend="add",
    seasonal=None
)
exp_fit = exp_model.fit()
print("\nExponential Smoothing applied successfully.")
print("\nFitted Values:")
print(exp_fit.fittedvalues)
```

```
Exponential Smoothing applied successfully.
```

Q21. Forecast the next 3 months

```

exp_forecast = exp_fit.forecast(3)
# Create next 3 month dates
future_dates = pd.date_range(
    start=monthly_df.index[-1] + pd.DateOffset(months=1),
    periods=3,
    freq="MS"
)
exp_forecast_df = pd.DataFrame({
    "Month": future_dates,
    "Forecast": exp_forecast.values
})
print("\nExponential Smoothing Forecast:")
print(exp_forecast_df)

```

```

Fitted Values:
Month
2025-01-01    121.743592
2025-02-01    128.365969
2025-03-01    134.988347
2025-04-01    141.610724
2025-05-01    148.233102
2025-06-01    154.855479
2025-07-01    161.477857
2025-08-01    168.100234
2025-09-01    174.722611
2025-10-01    181.344989
2025-11-01    187.967366
2025-12-01    194.589744
dtype: float64

```

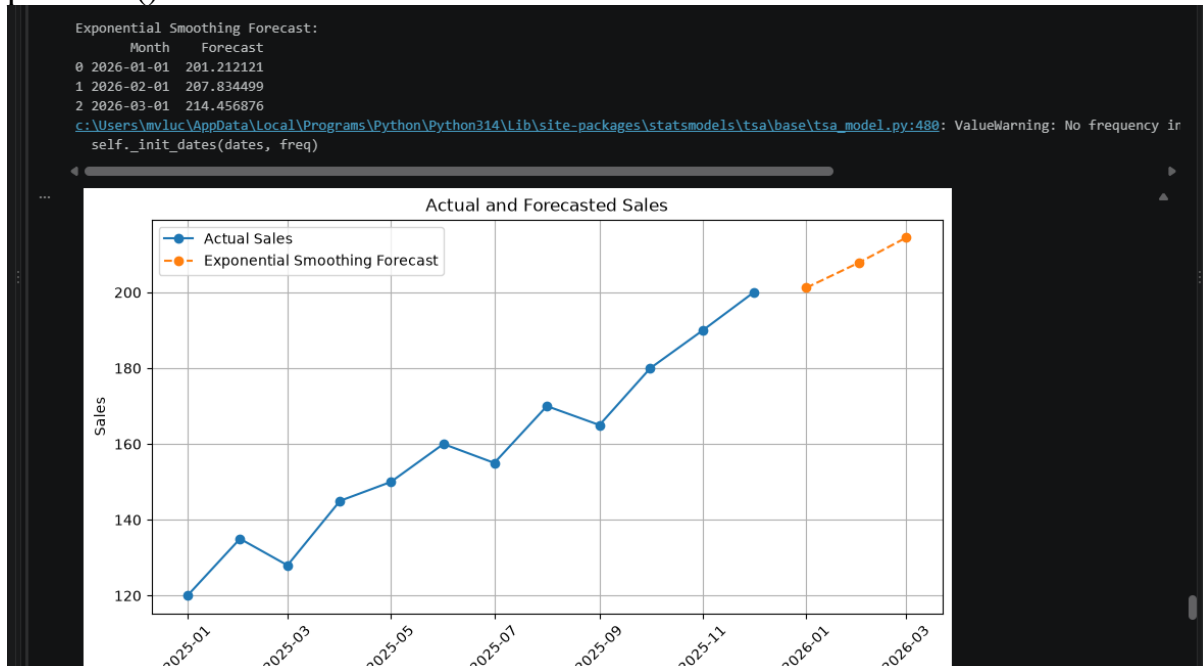
```

# Q22. Plot actual and forecasted sales
plt.figure(figsize=(10, 5))
# Actual sales
plt.plot(
    monthly_df.index,
    monthly_df["Sales"],
    marker="o",
    label="Actual Sales"
)
# Forecast
plt.plot(
    future_dates,
    exp_forecast.values,
    marker="o",
    linestyle="--",
    label="Exponential Smoothing Forecast"
)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Actual and Forecasted Sales")

plt.legend()

```

```
plt.grid(True)
plt.xticks(rotation=45)
plt.show()
```



Q23. Calculate forecasting error

```
exp_fitted = exp_fit.fittedvalues
exp_mse = mean_squared_error(
    sales_series,
    exp_fitted
)
exp_rmse = np.sqrt(exp_mse)
print("\nExponential Smoothing Forecast Error:")
print("MSE =", round(exp_mse, 4))
print("RMSE =", round(exp_rmse, 4))
```

```
Exponential Smoothing Forecast Error:
MSE = 26.0229
RMSE = 5.1013
```

Q24. Apply ARIMA to the same data

```
arima_model = ARIMA(
    sales_series,
    order=(1, 1, 1)
)
arima_fit = arima_model.fit()
print("\nARIMA Model Applied Successfully.")
print("\nARIMA Model Summary:")
print(arima_fit.summary())
```

```
ARIMA Model Applied Successfully.
```

Q25. Forecast next 3 months using ARIMA

```
arima_forecast = arima_fit.forecast(
```

```

    steps=3
)
arima_forecast_df = pd.DataFrame({
    "Month": future_dates,
    "Forecast": arima_forecast.values
})
print("\nARIMA Forecast:")
print(arima_forecast_df)

```

```

ARIMA Model Summary:
=====
SARIMAX Results
=====
Dep. Variable:      Sales      No. Observations:      12
Model:              ARIMA(1, 1, 1)  Log Likelihood      -40.405
Date:              Sat, 03 Oct 2026  AIC              86.811
Time:              09:35:39      BIC              88.005
Sample:            01-01-2025     HQIC             86.058
                  - 12-01-2025
Covariance Type:    opg

```

Q26. Compare ARIMA and Exponential Smoothing forecasts

```

forecast_comparison = pd.DataFrame({
    "Month": future_dates,
    "Exponential Smoothing": exp_forecast.values,
    "ARIMA": arima_forecast.values
})
print("\nForecast Comparison:")
print(forecast_comparison)

```

```

      coef  std err      z  P>|z|  [0.025  0.975]
-----
ar.L1    -1.0000    0.003  -296.271    0.000   -1.007   -0.993
ma.L1     0.9960    0.576    1.730    0.084   -0.133    2.125
sigma2    74.9840    0.008  9519.129    0.000   74.969   74.999

```

Q27. Calculate and compare forecasting errors

Exponential Smoothing error

```

exp_fitted = exp_fit.fittedvalues
exp_mse = mean_squared_error(
    sales_series,
    exp_fitted
)

```

```

exp_rmse = np.sqrt(exp_mse)

```

ARIMA fitted values

```

arima_fitted = arima_fit.fittedvalues

```

Remove missing values

```

valid_data = pd.concat(
    [
        sales_series.rename("Actual"),
        arima_fitted.rename("Predicted")
    ],
    axis=1
).dropna()

```

```

arima_mse = mean_squared_error(
    valid_data["Actual"],
    valid_data["Predicted"]
)
arima_rmse = np.sqrt(arima_mse)
# Compare errors
error_comparison = pd.DataFrame({
    "Model": [
        "Exponential Smoothing",
        "ARIMA"
    ],
    "MSE": [
        exp_mse,
        arima_mse
    ],
    "RMSE": [
        exp_rmse,
        arima_rmse
    ]
})

```

```

print("\nForecasting Error Comparison:")
print(error_comparison)

```

```

=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1          -1.0000         0.003   -296.271     0.000       -1.007       -0.993
ma.L1           0.9960         0.576     1.730     0.084       -0.133       2.125
sigma2         74.9840         0.008   9519.129     0.000       74.969       74.999
=====
Ljung-Box (L1) (Q):           1.19   Jarque-Bera (JB):           0.57
Prob(Q):                   0.27   Prob(JB):           0.75
Heteroskedasticity (H):       0.98   Skew:           0.40
Prob(H) (two-sided):         0.98   Kurtosis:         2.21
=====

```

```

# Q28. Identify model with lower forecasting error
print("\nModel with Lower Forecasting Error:")
if exp_rmse < arima_rmse:
    print("Exponential Smoothing has the lower RMSE.")
elif arima_rmse < exp_rmse:
    print("ARIMA has the lower RMSE.")
else:
    print("Both models have the same RMSE.")

```

```

...
5. Exponential Smoothing produced the
   lower forecasting error for this dataset.

```

```

# Q29. Final conclusion
print("FINAL CONCLUSION")
print("=====")

```

```
print("1. Monthly sales show an overall increasing trend.")
print("2. Exponential Smoothing was applied")
print("   to forecast the next 3 months.")
print("3. ARIMA was applied to forecast")
print("   the next 3 months.")
print("4. Both forecasting models were compared")
print("   using MSE and RMSE.")
if exp_rmse < arima_rmse:
    print("5. Exponential Smoothing produced the")
    print("   lower forecasting error for this dataset.")
elif arima_rmse < exp_rmse:
    print("5. ARIMA produced the lower forecasting")
    print("   error for this dataset.")
else:
    print("5. Both models produced the same error.")
print("\nPART B COMPLETED SUCCESSFULLY!")
```

```
...
5. Exponential Smoothing produced the
   lower forecasting error for this dataset.
```

```
...
5. Exponential Smoothing produced the
   lower forecasting error for this dataset.
```

```
PART B COMPLETED SUCCESSFULLY!
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)